

chapter 05

플러터는 크로스 플랫폼 프레임워크이다

기존 크로스 플랫폼 프레임워크와 차이점

react	flutter
코드를 네이티브 프레임워크로 전환하는 데 중점, 자바스크립트 브릿지를 거친다	스키아 엔진(오픈소스 2D 그래픽 렌더링 라이브러리)와 직접 통신

어디서든 일관된 UI를 제공할 수 있다

플러터 구조

1. 임베더 계층

6개의 플랫폼의 네이티브 플랫폼과 직접 통신을 하고 운영체제의 자체적 기능을 모듈화함
이 모듈들은 각 플랫폼의 네이티브 언어로 작성되어 있다

Render Surface Setup: 운영체제로부터 그림을 그릴 메모리 공간 할당 허락 받기

Native Plugins: GPS, 카메라, 배터리 상태 등 OS 고유 기능을 Dart 코드에서 쓸 수 있게 연결

App Packaging: 작성한 코드를 실제 스마트폰에 설치할 수 있는 형태(.apk, .ipa)로 묶어 주는 역할

Thread Setup: 운영체제에게 CPU 사용을 요청한다

Event Loop Interop: 사용자가 화면에 손을 대면 전기 신호가 발생하고 하드웨어가 이를 운영 체제로 전달한다. 이후 운영체제가 터치가 발생했다는 이벤트를 발생한다

2. 엔진 계층

Service Protocol: 외부 도구가 플러터 엔진 내부를 들여다보고 조작할 수 있게 해주는 통신 규약

Composition: 여러 층으로 겹쳐진 레이어(배경, 버튼, 텍스트 등)를 하나로 합쳐서 화면을 구성

Platform Channels: 닥트 코드에서 기기 자체의 기능(GPS, 카메라 등)이 필요할 때 네이티브 플랫폼에 요청을 전달하는 통로

Dart Isolate Setup: 닥트는 Isolate 라는 단위로 코드를 실행하는데 이를 생성하고 관리하여 앱이 멈추지 않고 여러 작업을 동시에 작업할 수 있게 돕는다.

Rendering: 닥트 코드에서 명령한 디자인 설계도를 실제 픽셀로 변환하여 화면에 그리는 핵심 역할

System Events: 운영체제로부터 오는 시스템 신호(배터리 부족, 화면 회전 등)를 앱 내부로 전달한다.

Dart Runtime Mgmt: 닥트 코드가 실행될 때 필요한 메모리를 할당하고 관리

Frame Scheduling/Frame Pipelining: 화면이 끊기지 않도록 초당 60~120 프레임의 속도로 그림을 그릴 순서와 시간을 관리한다

Asset Resolution: 이미지 파일, 글꼴 파일 등 앱에 포함된 리소스를 불러와서 엔진이 쓸 수 있게 준비한다.

Text Layout: 복잡한 텍스트(글꼴, 크기, 정렬 등)를 화면 어디에 어떻게 배치할 지 계산한다.

3. 프레임워크 계층

Material/Cupertino: 안드로이드와 iOS 디자인 스타일의 위젯을 제공

Widgets: 앱의 모든 구성 요소를 만드는 기본 단위

Rendering: 위젯의 크기와 위치를 결정하여 아래 엔진 계층으로 전달

Animation/Painting/Gestures: 화면의 움직임, 색칠, 사용자의 터치 입력 처리 담당

Foundation: 애니메이션, HTTP 통신 등 가장 기초적인 기능